Chapter 8 On-policy Prediction with Approximation

在本章中，我们开始考虑一种新的设定：将**价值函数 (Value function)** 表示为一种参数化的函数形式，而不是一个简单的表格。使用参数化函数来近似价值函数，使得我们能够处理那些**状态空间无限大**或**不可完全观测**的复杂环境。

价值函数逼近 (Value-function Approximation)

表格型方法的局限性 (Limitation of Tabular Methods)

在表格型方法中，价值函数 $v : \mathcal{S} \to \mathbb{R}$ 本质上是一个大小为 $|\mathcal{S}|$ 的表格，它为每个状态分配一个数值。这种**表格表示法 (tabular representation)** 有一个显著的局限性——它无法处理状态空间无限大的情况，而这在现实世界中经常发生。甚至在状态空间 $\mathcal{S}$ 包含的状态数量非常巨大时，表格表示法也是极其低效的。

- **概念解析：** 以前学的方法（如Q-learning的Q表）就像是“死记硬背”。有多少个状态，就建多大的表格。
- **通俗举例：** 假设你在玩一个赛车游戏。如果用表格法，你需要记录“在坐标(1,1)且速度为10时该怎么做”、“在坐标(1.1, 1)且速度为10.5时该怎么做”.....这种状态有无数个（连续空间），你的表格根本装不下，这就叫**维度灾难**。

参数化表示与泛化能力 (Parameterized Representation & Generalization)

这里，我们引入一种参数化表示，使用参数化形式 $\hat{v}(s, \mathbf{w})$ 来近似真实的价值函数，其中 $\mathbf{w} \in \mathbb{R}^d$ 是一个权重向量。通常，$\hat{v}$ 会被形式化地表示为基于状态特征的线性函数，或者是一个多层神经网络。ps.后面将会讲到。
请注意，在这种**参数化表示**下，仅仅改变一个权重的值，就会导致许多状态的价值发生改变。因此，当更新单一状态时，这种改变会从该状态**泛化 (generalize)**出去，从而影响许多其他状态的价值评估。（也就是增加了泛化的能力，不用手动一个个修改更新其他的状态价值，而是由机器进行修改）

概念解析： 我们不再用死表格，而是用一个“公式”或者“黑盒（如神经网络）”来算价值。你把状态 s 输入进去，它根据内部的权重 $\mathbf{w}$ 算出一个近似值。

通俗举例（泛化）： 假设你学会了开“丰田卡罗拉”（更新了大脑里的驾驶权重 $\mathbf{w}$）。当你坐进一辆“本田思域”（一个以前没见过的新状态 s）时，你依然能开走，因为两辆

车的特征（方向盘、油门、刹车）很相似。这就是**泛化能力**。参数化方法让你“触类旁通”，**更新一个经验，能造福无数个相似的状态**。同样修改一个状态，能影响其他相似的状态

函数逼近 (Function Approximation)

所有的预测方法都包含对估计价值函数的更新，这些更新会将特定状态的估计值推向某些**更新目标 (update targets)**。因此，我们可以用符号 $s \mapsto u$ 来表示单次更新，其中 s 是正在被更新的状态，u 是状态 s 的估计值要逼近的更新目标。例如，蒙特卡洛 (Monte Carlo) 的更新是 $s_t \mapsto g_t$；TD(0) 的更新是 $s_t \mapsto r_{t+1} + \gamma V(s_{t+1})$；多步TD (n-step TD) 的更新是 $s_t \mapsto g_{t:t+n}$

这三种更新方式，我们打个比方：足球比赛：

蒙特卡洛 (Monte Carlo): $s_t \mapsto g_t$
• **公式意义：** s_t 是当前状态，g_t 是直到回合结束时的**真实总回报**。你用最终的实际结果来更新当前的预测。
• **比喻：** 比赛开始了，你啥也不说，死死盯到 90 分钟全场比赛结束。如果最后比分是 3:1（总进球 4 个），你就把开场第 1 分钟的预测值往“4”这个真实结果去更新。
• **特点：** 最准确（因为没有猜的成分），但必须等整个回合结束才能更新，学习速度慢。**TD(0) (单步时序差分):** $s_t \mapsto r_{t+1} + \gamma V(s_{t+1})$
• **公式意义：** r_{t+1} 是走一步得到的真实奖励，$V(s_{t+1})$ 是你对下一个状态的**估计价值**（猜的）。
• **比喻：** 比赛进行到第 10 分钟，进了一个球（真实奖励 r_{t+1}）。你结合场上的最新局势（下一个状态 s_{t+1}），凭借自己的经验**预估**接下来还能进 2 个球（即 $V(s_{t+1})$）。于是你立刻把总进球预测更新为 $1+2=3$ 个。
• **特点：** 走一步就更新，学习效率极高。虽然有一部分是“猜的”（用估计更新估计，这叫 Bootstrapping），但通常能更快收敛。**多步 TD (n-step TD):** $s_t \mapsto g_{t:t+n}$
• **公式意义：** 往后走 n 步，拿到 n 步的真实奖励，然后再用第 n 步的估计值来补全剩下的部分。
• **比喻：** 你觉得看 90 分钟太久（MC），看 1 分钟又太短容易误判（TD(0)）。于是你决定看 30 分钟（n步）。看完 30 分钟后，根据这段时间的真实进球数，加上你对剩下 60 分钟的预估，来更新你开局的预测。
• **特点：** 它是 MC 和 TD(0) 之间的折中和过渡。

基于这种理解，当我们使用参数化形式 $\hat{v}(s, \mathbf{w})$ 时，每一次更新本质上就是让该状态下的函数值去拟合给定的目标。这个过程通常被称为**函数逼近 (function**

approximation)。在机器学习的语境下，这可以通过**监督学习 (supervised learning)** 来完成。我们将每次更新 $s \mapsto g$ 视为一个传统的训练样本，这就让我们能够利用大量现有的函数逼近方法来进行价值预测。

概念解析： 强化学习的更新过程，其实就是把它伪装成“监督学习”。状态 s 就是“考试题目”（输入特征 X），更新目标 u 就是“标准答案”（标签 Y）。我们不断调整神经网络的权重 $\mathbf{w}$，让模型给出的答案 $\hat{v}$ 越来越接近标准答案 u。也就是将S的V拟合到更真实推导出的。

Note：强化学习与监督学习的本质区别

强化学习和监督学习之间的一个关键区别在于：在强化学习中，学习过程是**在线 (online)** 发生的，即智能体一边与环境交互一边学习。因此，强化学习通常要求函数逼近方法能够处理**非平稳的目标函数 (nonstationary target functions)**。具体来说，当策略 π 在被持续更新时，我们同时还在试图学习该策略下的价值函数 v_π 或 q_π。

- **在线学习 (Online Learning)：** 数据是源源不断流进来的，而不是像监督学习那样，一开始就把所有题库（数据集）都给你准备好了。
- **非平稳目标 (Nonstationary target)：**
  - **监督学习（平稳）：** 像是在打固定的靶子。图片里是一只猫，标签永远是“猫”，不会变。
  - **强化学习（非平稳）：** 像是在**站在移动的船上射击移动的飞碟**。因为你的智能体在学习，它的策略（行为方式）在不断改变（比如它变聪明了，存活时间更长了）；因为策略变了，对应状态的“真实价值”也就跟着变了。所以你的神经网络不仅要追赶目标，目标本身还在不断移动。

对于非平稳目标：你更新了一次参数 $\mathbf{w}$，你的预测能力变强了一点；但同时，你用来作为“标准答案”的那个预估值 $\hat{V}(s_{t+1}, \mathbf{w})$ 也跟着变了！靶心自己长了腿在跑。

怎么训练？岂不是崩溃了？

是的，早期用神经网络做强化学习时，经常因为这个原因导致训练崩溃（不收敛）。后来学者们想出了一些聪明的办法来对付这种非平稳性，最著名的就是 DeepMind 在 DQN 算法中引入的 **目标网络 (Target Network)**：

• **解决思路：** 既然靶心一直在动，那我们就**把靶心按住一会**。

• 具体做法是：准备**两个**一模一样的神经网络。一个是“主网络”，一直在更新；另一个是“目标网络”，专门用来产生目标标签。我们把“目标网络”冻结住（比如 1000 步内不改变参数）。这 1000 步里，目标（标签）就是固定的（变成了平稳的），主网络就可以安心去拟合。等 1000 步到了，再把主网络的参数复制给目标网络，更新一次靶

心的位置。
这样既保留了 TD 走一步学一步的效率，又解决了目标乱跑导致无法训练的问

预测目标 (The Prediction Objective)

在上一节我们知道要用参数化的函数（比如神经网络）来代替表格记录价值。这一节的核心问题是：既然是“近似”，我们就需要一个指标来衡量这个近似函数到底“好不好”。这就是我们要设定的**预测目标（目标函数）**。

均方误差 (Mean Squared Error, MSE)

函数逼近通常被公式化为一个优化问题，目的是最小化某个特定的损失函数。**均方误差**是函数逼近中最广泛使用的损失函数。设 $f : \mathcal{X} \to \mathbb{R}^d$ 是一个由权重向量 $\mathbf{w}$ 参数化的实值函数。给定一组输入-输出对 $\{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}$，关于 $\mathbf{w}$ 的均方误差 (MSE) 定义为：

$$\mathrm{MSE}(\mathbf{w}) \doteq \frac{1}{n} \sum_{i=1}^{n} \| f(\mathbf{x}_i, \mathbf{w}) - y_i \|^2 \quad \text{— (8.1)}$$

概念解析：这是传统机器学习中最常见的目标。预测的值 $f(\mathbf{x}_i, \mathbf{w})$ 和真实的标签 y_i 之间差了多少，平方一下，然后求个平均。数值越小，说明你的模型预测越准。

公式符号：
• $f(\mathbf{x}_i, \mathbf{w})$：模型的预测结果。把第 i 个输入数据 $\mathbf{x}_i$ 喂给带有参数 $\mathbf{w}$ 的模型 f，算出来的结果
• y_i：标准答案（真实目标值）。
• $\mathbf{w}$：模型的参数（比如神经网络的权重），我们就是要找出一组最好的 $\mathbf{w}$ 让 MSE 最小。

$\mathcal{X}$：代表**输入空间**。在强化学习里，这通常就是所有的状态构成的集合（比如迷宫的所有坐标，或者棋盘的所有合法局面）。

$\mathbb{R}^d$：$\mathbb{R}$ 代表实数集，右上角的 d 代表维度。
• 如果 $d = 1$，也就是 $\mathbb{R}^1$（通常直接写成 $\mathbb{R}$），意味着输出是一个**单一的实数**。在强化学习里，这对应着计算状态价值 $V(s)$，即“这个状态有多好”。
• 如果 $d > 1$，意味着输出是一个包含 d 个实数的**向量**。在强化学习里，这通常对应着计算动作价值 $Q(s, a)$。比如在这个状态下有 4 个可选动作，网络就会一次性输出 4 个值，此时 $d = 4$。

均方价值误差 (Mean Squared Value Error, $\overline{\mathrm{VE}}$)

在强化学习的语境下，我们面临的情况通常是：状态的数量远远多于参数的数量（甚至有无限多个状态），因此通常不可能完全精确地匹配真实的价值函数。在这些情况下，让某个状态的估计变得更准确，往往会导致其他状态的估计变得不够准确。
因此，我们需要通过给出一个状态空间 $\mathcal{S}$ 上的状态分布 μ，来指定哪些状态更加重要。该分布需满足 $\mu(s) \geq 0, \forall s \in \mathcal{S}$ 且 $\sum_{s \in \mathcal{S}} \mu(s) = 1$。取值在状态空间，期望概率大于0，且概率和为1

每个状态的*误差*定义为近似价值 $\hat{v}(s, \mathbf{w})$ 与真实价值 $v_\pi(s)$ 之间的差异。使用分布 μ 对整个状态空间进行加权，我们可以得到**均方价值误差 (Mean squared value error)**，记作 $\overline{\mathrm{VE}}$，其正式定义为：

$$\overline{\mathrm{VE}}(\mathbf{w}) \doteq \sum_{s \in \mathcal{S}} \mu(s)[\hat{v}(s, \mathbf{w}) - v_\pi(s)]^2 \quad \text{— (8.2)}$$

这个指标的平方根被称为 root $\overline{\mathrm{VE}}$，它可以粗略地衡量近似值与真实值之间的差异程度。

- **核心痛点（为什么要加权？）**：因为我们的模型（比如一个小神经网络）参数有限，相当于“脑容量有限”，记不住所有状态的准确价值。就像一张小尺寸的毯子盖不住整个人，盖住了头就会漏出脚。顾此失彼是常态。
- **通俗举例**：既然毯子不够大，我们就得决定“保头还是保脚”。状态分布 $\mu(s)$ 就是“重要性权重”。在游戏里，有些状态玩家经常遇到（比如超级玛丽的第一关起点），这种状态的 $\mu(s)$ 就很大，必须算得准一点；有些状态几乎一辈子遇不到（比如卡进了地图某个死角 bug），它的 $\mu(s)$ 就很小，算错了也无所谓。
- **公式符号**：
  - $\mu(s)$：状态 s 出现的概率或者重要性权重。
  - $\hat{v}(s, \mathbf{w})$：我们的模型算出来的近似价值。
  - $v_\pi(s)$：在策略 π 下，该状态真正的理论价值。

Notice：需要牢记的是，在强化学习中，拟合价值函数并不是最终目标。为了找到最优策略，最适合的价值函数，不一定就是那个能把均方价值误差 ($\overline{\mathrm{VE}}$) 降到最低的价值函数。

通俗解析：强化学习的最终目的是**做决策（选出最好的动作）**，而不是**当预言家（精准预测具体分数）**。**举个例子**：假设面前有两条路，真实价值分别是 100 分和 10 分。
- *模型 A*：预测两条路分别是 99 分和 9 分。（误差很小）
- *模型 B*：预测两条路分别是 1000 分和 -500 分。（误差极大）
- 从选动作的角度看，模型 A 和模型 B 都会让你选择第一条路。所以哪怕模型 B 的误差 ($\overline{\mathrm{VE}}$) 很大，但在推导最优策略上，它和模型 A 一样好。

同策略分布 (On-policy Distribution)

状态概率 $\mu(s)$ 通常被选择为“在状态 s 中停留的时间比例”。在同策略（on-policy）训练下，这被称为**同策略分布**。
针对任务类型的不同，同策略分布 $\mu(s)$ 的计算也有所区别

- s：当前我们正在关注的状态（比如棋盘上的某个特定残局）。
- s'：上一个状态（也就是来到 s 之前，我们在哪里）。
- $\mu(s)$：代表状态 s 的**平稳分布概率**。通俗说，就是如果你在这个环境里瞎逛或者按某种策略一直走，你有百分之几的时间是待在状态 s 的。
- $P_\pi(s|s')$：在策略 π 的指导下，从状态 s' 转移到状态 s 的**总概率**。
- a：动作。
- $\pi(a|s')$：你的**策略**。即在状态 s' 时，你选择执行动作 a 的概率。
- $p(s|s',a)$：环境的**状态转移概率**。意思是，在状态 s' 执行了动作 a 后，环境把你扔到状态 s 的概率。
- $\eta(s)$：（读作 eta）在“回合制”任务（比如一局游戏）中，一局下来你**平均会访问状态 s 几次**。
- $h(s)$：一局游戏**刚好从状态 s 开始**的概率。

- **连续性任务 (Continuing Tasks)**

对于连续性任务，同策略分布指的是在策略 π 下的**平稳分布 (stationary distribution)**：

$$\mu(s) = \sum_{s'} \mu(s') P_\pi(s|s') = \sum_{s'} \mu(s') \sum_a \pi(a|s') p(s|s',a), \quad \forall s \in \mathcal{S} \quad \text{— (8.3)}$$

连续性任务就是永远不会结束的游戏/系统。平稳分布意味着，如果系统运行了无限长时间，你在任意时刻“按下暂停键”，发现系统刚好处于状态 s 的概率就是 $\mu(s)$。公式的意思是：流出状态 s 的概率等于流回状态 s 的概率（达到了动态平衡）。想象一个水池系统，水在各个池子（状态）之间流来流去。$\mu(s)$ 就是水池 s 里的水位线。当水流稳定后，流进这个水池的水，必须等于这个水池现有的水

▼ 如何详细理解

触及到了马尔可夫链里的核心概念：**平稳分布 (Stationary Distribution)** 或 **全局平衡 (Global Balance)**。

直觉解释：

当你的一个宠物猫刚开始的前几分钟，它可能一直在键盘上踩来踩去，这时候状态的分布是不稳定的。但是，当它在桌子上运行了几天、几个月之后（无限长的时间），它在各个地方待的时间比例会**趋于一个稳定的常数**。比如，它总会有 60% 的时间在睡觉，30% 在键盘上，10% 在边缘试探。

这个稳定的状态，就是“平稳分布”。**在这个平稳状态下，必然发生一件事：水池 B（键盘）的水位不再变化了！**

水位不变化意味着什么？意味着**“流进来的水”必须等于“流出去的水”**。这就是为什么公式 8.3 可以画等号的原因：

$$\text{状态 B 的当前概率水量 } \mu(s) = \text{从四面八方流入状态 B 的概率水量总和 } \sum_{s'} \mu(s') P_\pi(s|s')$$

如果流入大于流出，

水池 B 的概率就会增加；如果流出大于流入，水池 B 的概率就会减少。**只有当流入完全等于流出时，这个概率 $\mu(s)$ 才会稳定下来，成为我们最终需要的那个“平稳分布概率”。**

这就是数学家处理“无限循环”问题的终极武器：我们不去一步步算它到底走到了哪，而是直接假设它达到了最终的“动态平衡”状态，然后利用“流入=流出”这个平衡方程，把结果给解出来。

推导：

你现在处在状态 s 的概率（左边的 $\mu(s)$），等于**“你之前在所有可能的状态 s' 的概率”** 乘以 **“从 s' 走到 s 的概率”**，然后把它们全部加起来（求和符号 $\sum_{s'}$）。那从 s' 怎么走到 s 呢？这取决于两步：首先你得根据策略决定选哪个动作（$\pi(a|s')$），然后环境还得配合你，把你送到 s（$p(s|s', a)$）。把这两步乘起来再对所有动作求和，就是第二部分的展开。

- **幕式任务 (Episodic Tasks)**

对于幕式任务，同策略分布略有不同，因为它依赖于初始状态是如何选择的。令 $h(s)$ 表示初始状态的分布，$\eta(s)$ 表示在一个单一的回合（episode）中，平均在状态 s 停留的时间步数。那么我们有：

$$\eta(s) = h(s) + \sum_{s'} \eta(s') \sum_{a} \pi(a|s') p(s|s', a), \quad \forall s \in \mathcal{S} \quad \text{— (8.4)}$$

通俗解析： 假设你要计算在一局游戏里，你踩到陷阱格子（状态 s）的平均次数。你怎么踩到陷阱的？只有两种可能：
1. **开局就出生在陷阱上**：这就是 $h(s)$。
2. **从别的地方走过来的**：这部分的逻辑和 8.3 一模一样。你先算出你访问其他所有格子 s' 的次数 $\eta(s')$，然后乘以从 s' 走到陷阱 s 的概率。把"出生在这里的次数"加上"从别处走过来的次数"，就是一局游戏里你总共经过这里的次数！

可以通过求解上述方程组得出期望的访问次数 $\eta(s)$。然后，通过对 $\eta(s)$ 进行归一化，就可以推导出同策略分布：

$$\mu(s) = \frac{\eta(s)}{\sum_{s' \in \mathcal{S}} \eta(s')}, \quad \forall s \in \mathcal{S} \quad \text{— (8.5)}$$

概念解析：幕式任务是指有明确起点和终点的任务（像一局王者荣耀，或者下完一盘国际象棋）。**通俗举例与公式符号**：
- $h(s)$：**出生点概率**。即一局游戏开始时，你出生在状态 s 的概率。
- $\eta(s)$：**总踩坑次数**。一局游戏从头打到尾，你平均会经过状态 s 多少次。公式 8.4 就是在算：`总次数 = （如果是出生点就记作1） + （从其他所有状态转移到s的次数总和）`。
- 归一化（公式 8.5）：既然我们要的是**概率分布**（总和必须为1），那我们只需要把"经过状态 s 的平均次数"，除以"一局游戏走过的总步数（所有状态访问次数之和）"，就算出了在状态 s 停留的时间比例 $\mu(s)$ 了。

最后，我们要达成一个目的：**在特定的 AI 策略下，地图上的每一个角落，AI 到底有多大的概率会去踩一脚？**
踩得多的地方，$\mu(s)$ 就大，我们在用神经网络拟合（训练）的时候，这里的权重就高，MSE 惩罚就重；踩不到的地方，$\mu(s)$ 接近 0，网络预测准不准也就无所谓了。

随机梯度与半梯度方法 (Stochastic-gradient and Semi-gradient Methods)

在前一节中，我们定义了衡量预测好坏的标准（均方误差 $\overline{\mathrm{VE}}$）。既然有了目标，这一节我们要讨论的就是：**如何通过算法来调整参数（权重 $\mathbf{w}$），从而让这个误差尽可能小**。这里介绍的是基于随机梯度下降的一类学习算法。

随机梯度下降 (SGD)

是一种用于优化给定目标函数的迭代算法，该目标函数由多个项组成，每一项都依赖于一个样本。
具体来说，考虑如下定义的关于参数 $\boldsymbol{\theta}$ 的目标函数 $F:\Theta\to\mathbb{R}$：

$$F_{\mathcal{D}}(\boldsymbol{\theta})=\mathbb{E}_{x\sim\mathcal{D}}[f(x,\boldsymbol{\theta})]\triangleq\frac{1}{n}\sum_{i=1}^{n}f(x_i,\boldsymbol{\theta}),\quad -(8.6)$$

- $\boldsymbol{\theta}$ **(Theta)**：模型的参数（在神经网络里就是所有连接权重的总和，相当于后面强化学习里的 $\mathbf{w}$）。
- $\mathcal{D}$ **(Distribution)**：数据的真实概率分布。
- $\mathbb{E}$ **(Expectation)**：数学期望（也就是加权平均值）。
- $\nabla_{\boldsymbol{\theta}}$ **(Nabla)**：梯度（对参数求导，指示了“下山”最快的方向）。
- α_t **(Alpha)**：第 t 步的学习率（步长）。

在理想状态下，你想算模型的平均误差（左边），你需要知道数据的全部概率分布然后求期望。但这不可能做到。所以我们用**“经验平均”**（右边）来代替：随便抓 n 个样本出来，算算这 n 个样本的平均误差就行了。

其中 $\boldsymbol{\theta}$ 是我们想要优化的参数，$\mathcal{D}=\{x_1,\ldots,x_n\}$ 是一个样本集合，集合中的每个样本对应目标函数中的一项。那么 F 关于 $\boldsymbol{\theta}$ 的梯度为：

$$\nabla_{\boldsymbol{\theta}}F(\boldsymbol{\theta})=\mathbb{E}_{x\sim\mathcal{D}}[\nabla_{\boldsymbol{\theta}}f(x,\boldsymbol{\theta})]=\frac{1}{n}\sum_{i=1}^{n}\nabla_{\boldsymbol{\theta}}f(x_i,\boldsymbol{\theta}).\quad -(8.7)$$

“怎么修改参数才能让总体误差最小？”（求梯度的期望）。数学上证明了，这等于“算出每个样本误差的梯度，然后求平均”。

我们可以看到，计算这个梯度需要对整个样本集 $\mathcal{D}$ 中所有独立项的梯度求平均，当样本数 n 很大时，这种计算的代价会变得极其昂贵（令人望而却步）。

相反，*随机梯度下降*在每一步更新时，只使用**其中一个样本的梯度**来代替平均梯度：

$$\boldsymbol{\theta}_{t+1} \leftarrow \boldsymbol{\theta}_t - \alpha_{t+1} \nabla_{\boldsymbol{\theta}} f(x_{i(t+1)}, \boldsymbol{\theta}), \quad -(8.8)$$

其中 $i(t+1)$ 是在第 $(t+1)$ 次迭代时选中的样本索引，α_{t+1} 是那次迭代的步长（学习率），通常设为 $\alpha_t = \alpha_1/t$。
与标准的梯度下降相比，*随机梯度下降*的移动速度快得多，但它允许在每个时间步上存在一定程度的随机性。

连 n 个样本的平均我都懒得算！我每次**只抽 1 个**随机样本 $x_{i(t+1)}$，算它的梯度，然后让参数 $\boldsymbol{\theta}$ 减去这个梯度（乘以一个学习率 α）。这就好比你蒙着眼睛下山，不想花时间用雷达扫描周围地形，而是随便伸脚试探一下哪边往下倾斜，就往哪边迈一步。虽然每一步可能走偏，但由于样本是随机抽的，长期来看你依然会晃晃悠悠走到谷底。

概念解析：

- **梯度 (Gradient)**：简单来说，就是函数增长最快的方向。在误差函数里，我们想要误差变小，所以我们要**逆着**梯度的方向走（减去梯度）。
- **标准梯度下降 (批量梯度下降)**：为了找准下山的路，你要把所有的地形数据（所有样本）都看一遍，算出一个平均的下山方向。如果数据有几百万条，算一次方向要很久。
- **随机梯度下降 (SGD)**：你不想等了，随便抽一条数据（一个样本），算一下这条数据告诉你的下山方向，然后直接迈一步。**通俗举例**：假设你想调查全国人民对某款饮料的喜爱程度（找最优配方）。
  - *标准梯度下降*相当于：你每次调整配方前，都要发问卷给全国14亿人，等所有人填完算个平均分，再决定加糖还是减糖。太慢了！
  - *随机梯度下降*相当于：你在街上随便拉一个人尝一口，他说“太甜了”，你马上就把总配方里的糖减一点。虽然这个人的口味不能代表全国，但只要你不断地在街上拉不同的人尝，并且每次步子迈小一点，最后配方还是会趋近于大众口味，而且调整速度极快。
  - **公式符号**：
    - $\boldsymbol{\theta}$：模型的参数（在强化学习里就是权重 $\mathbf{w}$）。
    - α_{t+1}：步长/学习率。控制你这一步迈多大。如果 $\alpha_t = \alpha_1/t$，意味着步子会随着时间越来越小，有助于最后稳定在谷底。

SGD 的收敛性 (Convergence of SGD)

一般来说，研究表明如果目标函数 F 是**强凸的 (strongly convex)**，并且步长 α_t 满足以下条件：

$$\sum_{t=1}^{\infty} \alpha_t = \infty \quad \text{并且} \quad \sum_{t=1}^{\infty} \alpha_t^2 < \infty, \quad -(8.9)$$

那么*随机梯度下降*将几乎必然收敛到**全局最优解**。注意，当步长设为 $\alpha_t = \alpha_1/t$ 时，它们满足上述条件。打个比方第 1 步：学习率 0.1 / 1 = 0.1；第 100 步：学习率 0.1 / 100 = 0.001。为什么这么设定： 随着你越来越接近碗底（最优解），你需要变得极其谨慎。如果你一直保持 0.1 的大步子，你就会在碗底两边反复横跳，永远停不下来。除以 t 保证了它最终会完美收敛。

- 条件1 $\sum \alpha_t = \infty$：保证步长加起来能让你走到任何地方（不会因为步子太早变小而半路停滞）。
- 条件2 $\sum \alpha_t^2 < \infty$：保证步长最终会趋近于 0（走到谷底时能停稳，不会在谷底反复横跳）。

你的步子必须越迈越小。当你接近谷底时，如果步子还很大，你就会在谷底两边反复横跳，永远停不下来。所以随着时间推移，学习率 α 必须逐渐衰减到 0。

什么是强凸的 (Strongly Convex)？

• **通俗比喻**：想象你要把一个玻璃球滚到碗底。

◦ 普通的“凸函数”就像一个普通的平底大碗，球滚到底部某处就停了，但底部可能是一大片平地，最小值有很多个。

◦ **强凸函数**就像一个极其完美的抛物线型漏斗（比如二次函数 $y = x^2$）。无论你把球丢在哪，它都会以一种“被强烈吸引”的姿态，极其确定地滚向唯一的最底端（全局唯一最优解）。

• **在 RL 里的意义**：我们在做线性函数逼近时，损失函数 $\overline{VE}$ 是一个**强凸函数**。这意味着我们用梯度下降去寻找最优权重 $\mathbf{w}$ 时，绝对不会卡在半山腰（局部最优），一定能找到全宇宙最完美的那个解！

价值预测的随机梯度方法 (SGD for Value Prediction)

回到价值预测的问题，我们想要最小化关于权重向量 $\mathbf{w}$ 的均方价值误差 ($\overline{\mathrm{VE}}$)：

$$\overline{\mathrm{VE}}(\mathbf{w}) = \sum_{s \in \mathcal{S}} \mu(s)[v_\pi(s) - \hat{v}(s, \mathbf{w})]^2 = \mathbb{E}_{s \sim \mu}[(v_\pi(s) - \hat{v}(s, \mathbf{w}))^2]. \quad \text{— (8.10)}$$

人话就是1.把概率分布变成随机选取的S的出现期望；2.把VE和梯度下降结合

使用 *SGD*，对于权重 $\mathbf{w}$ 的每一步更新可以这样计算：

$$\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t - \frac{1}{2}\alpha \nabla_{\mathbf{w}}[v_\pi(s_t) - \hat{v}(s_t, \mathbf{w}_t)]^2$$

$$= \mathbf{w}_t + \alpha[v_\pi(s_t) - \hat{v}(s_t, \mathbf{w}_t)]\nabla_{\mathbf{w}}\hat{v}(s_t, \mathbf{w}_t). \quad \text{— (8.11)}$$

省略分布 $\mu(s)$：因为我们在同策略 (on-policy) 下生成样本，状态 S_t 天然就是按照分布 μ 出现的。所以我们直接拿当前遇到的状态 s_t 更新就行，不需要在公式里显式地乘以 $\mu(s)$。我们根本不需要傻乎乎地去算那个复杂的 $\mu(s)$ 分布表！只要让 AI 智能体在环境里真实地玩游戏、瞎逛，它**自然而然**访问各个状态的频率，就是符合 $\mu(s)$ 的！我们只要把它玩游戏收集到的数据存进“经验回放池”，然后从中随机抽样训练，这就完美等价于按照 $\mu(s)$ 进行了加权！

常数步长 α：在强化学习中，目标通常是**非平稳的**（我们在追踪一个移动的靶子），所以通常使用固定的常数步长 α，而不是逐渐减小的步长（不希望后来失去学习能力）。

目标值未知：真实的价值 $v_\pi(s_t)$ 在现实中是未知的。我们必须用一个近似的方法来构造目标输出 U_t（比如蒙特卡洛回报或 TD 目标）。用 U_t 替换 $v_\pi(s_t)$，得到通用的更新公式：

$$\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \alpha[U_t - \hat{v}(s_t, \mathbf{w}_t)]\nabla_{\mathbf{w}}\hat{v}(s_t, \mathbf{w}_t). \quad \text{— (8.12)}$$

公式通俗理解：`新权重 = 老权重 + 步长 * (目标值 - 预测值) * 预测值对权重的敏感度(梯度)`。如果预测偏低了（目标 > 预测），公式会让权重朝着使预测值增大的方向调整

梯度蒙特卡洛算法 (Algorithm 8.1 Gradient Monte Carlo

如果我们的目标 U_t 选择的是蒙特卡洛的回报 G_t。因为 G_t 是真实价值 $v_\pi(s)$ 的**无偏估计 (unbiased estimate)**，所以用 G_t 代替 U_t 可以保证算法在满足步长条件时收敛到局部最优。

Algorithm 8.2 Semi-gradient TD(0) for value prediction

```
Initialize weight vector w ∈ ℝ^d arbitrarily
for each episode do
    Choose initial state s_0
    for each time step t = 0, 1, ..., T − 1 do
        Choose an action a_t ~ π(·|s_t)
        Take action a_t, and observe r_{t+1} and s_{t+1}
        Let u_t = r_{t+1} + γ v̂(s_{t+1}, w)
        Update w ← w + α (u_t − v̂(s_t, w)) ∇_w v̂(s_t, w)
    end for
end for
return w
```

半梯度方法 (Semi-gradient Method)

当更新目标 U_t 使用**自举法 (bootstrapping)** 时，问题会变得稍微复杂一些，因为这个目标本身在一定程度上也依赖于当前的权重向量 $\mathbf{w}$。例如，在 $TD(0)$ 中，更新目标是：

$$u_t = r_{t+1} + \gamma \hat{v}(s_{t+1}, \mathbf{w}).$$

尽管如此，我们仍然可以使用公式 (8.12) 中的更新规则，而**不去考虑 $\mathbf{w}$ 对目标产生的影响**。这种方式被称为**半梯度方法**。

概念解析：

- **真·梯度**：如果要对 TD 目标求导，因为目标里包含 $\gamma\hat{v}(s_{t+1}, \mathbf{w})$，它里面也有 $\mathbf{w}$，所以严格的数学求导必须对这部分也求梯度。
- **半梯度**：我们“偷个懒”，把目标 U_t 当作一个固定的常数（假装它里面没有 $\mathbf{w}$），求导的时候只对当前状态的预测值 $\hat{v}(s_t, \mathbf{w})$ 求导。这就只求了“一半”的梯度，所以叫半梯度。

通俗举例：假设你要预测明天和后天的股票价格（当前权重决定预测算法）。你今天的预测 = 明天的预测 + 差价。

- *全梯度*：你调整预测算法时，考虑到算法一变，你对“明天”的预测也会跟着变，极其复杂。
- *半梯度*：你假设“我对明天的预测已经是既定事实，是个固定数字”，然后只调整算法让今天的预测逼近这个固定数字。

补充

例子：预测马拉松完赛时间
假设你是一个 AI 教练，你的大脑里有一套评估模型（权重 $\mathbf{w}$），用来预测选手跑完全程还需要多久。
1. **当前状态 s_t（跑到了第 10 公里）：**
你用大脑（模型 $\mathbf{w}$）估算了一下，选手还需要 **120 分钟** 跑完。这就是 $\hat{v}(s_t, \mathbf{w})$。
2. **过了一会儿（跑到了第 11 公里）：**
你掐了下秒表，这 1 公里真实花了 **5 分钟**（这就是真实奖励 R_{t+1}）。
此时，你再用**同一个大脑**（同一个模型 $\mathbf{w}$）评估一下从第 11 公里跑到终点还需要多久。你脑子一转，估算是 **114 分钟**（这就是 $\hat{v}(s_{t+1}, \mathbf{w})$）。
3. **构建目标 U_t：**
根据 TD(0) 逻辑，你现在有了一个更靠谱的完赛时间目标：`5 分钟（真实）+ 114 分钟（最新估算）= 119 分钟`。这就是 U_t。
4. **计算误差并更新：**
你发现刚才在第 10 公里猜的 120 分钟偏慢了（比目标 119 分钟多了 1 分钟）。于是你要修改你大脑的评估逻辑（更新权重 $\mathbf{w}$）。
真正的矛盾来了：
注意到了吗？你的目标 U_t（119 分钟）里，包含了你在第 11 公里的估算（114 分钟）。而这个估算是由你的**当前权重 $\mathbf{w}$** 算出来的！
- **如果用“真正的全梯度下降”：** 数学上，当你试图修改 $\mathbf{w}$ 去逼近 119 时，随着你大脑逻辑 $\mathbf{w}$ 的改变，你对第 11 公里的估算（114）**也会跟着变**！目标 U_t 就像一个长了腿的活物，你往前追一步，它也往前跑一步。如果要严谨地求导，你必须把“目标随权重变化”的连锁反应也算进去，数学公式会变得极其庞大、复杂，甚至无法计算。
- **“半梯度 (Semi-gradient)”的流氓妥协：** 数学家们一拍大腿说，算了！我们在求导数、算梯度的那一瞬间，**强行把目标 U_t 冻结住！** 我们不管 U_t 里面是不是也包含了 $\mathbf{w}$，我们直接把 U_t 当作一个天上掉下来的、绝对不会变的常数（比如就把 119 钉死在那里）。我们只对当前的估算 $\hat{v}(s_t, \mathbf{w})$ 求导。
因为我们只对公式里**一半**的 $\mathbf{w}$（预测值里的）求了梯度，而忽略了**另一半** $\mathbf{w}$（目标值里的），所以它叫**半梯度 (Semi-gradient)**。
总结
半梯度方法之所以伟大，是因为它极其高效。虽然数学上它是个“残缺”的梯度，不能保证像全梯度那样 100% 完美收敛到全局最优点，但在实际应用中（尤其是在线性函数逼近下），它不仅计算飞快，而且总能收敛到一个非常接近最优解的地方。

半梯度方法的优缺点 (Note)

- **缺点**：因为不是真正的梯度，半梯度方法收敛起来不如纯梯度方法那么稳健（在非线性网络下有时会发散）。

- **优点**：在某些情况（如线性函数）下能可靠收敛；**更重要的是，它们学习速度快得多**，并且使得在线的、持续的自我更新成为可能（不需要像蒙特卡洛那样等一局游戏结束）。

也就是在求梯度时，**我们假装 U_t 里面的 $\mathbf{w}$ 是个死数字（常数）**，不让它参与求导。我们只对后面的那个预测值 $\hat{v}$ 求导。这种“算一半留一半”的流氓做法，就叫半梯度。它不严谨，但在实践中极其高效。

线性方法 (Linear Methods)

虽然近似价值函数 $\hat{v}(s, \mathbf{w})$ 可以是各种复杂的数学形式（比如深度神经网络），但有一种特别重要且基础的特殊情况，那就是**线性函数**。

近似价值函数的一个特别重要的形式是**线性函数 (linear function)**：

$$\hat{v}(s, \mathbf{w}) = \mathbf{w}^T \mathbf{x}(s) = \sum_{i=1}^{d} w_i x_i(s)$$

其中 $\mathbf{x}(s)$ 是状态 s 的向量化表示，通常被称为**特征向量 (feature vector)**，而 $x_i(s)$ 是 $\mathbf{x}(s)$ 的第 i 个分量。
在使用线性函数的情况下，近似价值函数 $\hat{v}$ 对权重向量 $\mathbf{w}$ 的梯度就是特征向量本身：

$$\nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w}) = \mathbf{x}(s)$$

因此，上一节讲到的 SGD 更新公式可以简化为一个特别简单的形式：

$$\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \alpha(U_t - \hat{v}(s_t, \mathbf{w}))\mathbf{x}(s_t) \quad \text{— (8.14)}$$

概念解析：线性方法的核心思想是：状态的价值，等于**各个特征的值**乘以**对应的权重**，然后求和。

通俗举例：假设你在评估一套**二手房的价值 ($\hat{v}$)**。
- **特征向量 $\mathbf{x}(s)$**：房子的具体属性，比如 $\mathbf{x} = [\text{面积}, \text{房龄}, \text{距离地铁距离}]$。
- **权重向量 $\mathbf{w}$**：市场对这些属性的定价，比如 $\mathbf{w} =$

$[\text{每平米}5\text{万}, \text{每年折旧减}2\text{万}, \text{每远}1\text{公里减}10\text{万}]$。
• **线性求和**：房子的总估价 = $5 \times \text{面积} + (-2) \times \text{房龄} + (-10) \times \text{距离}$。**为什么梯度就是 $\mathbf{x}(s)$？**：在多项式 $w_1x_1 + w_2x_2$ 中，对 w_1 求导，结果自然就是常数项 x_1。这让更新公式变得极其简单：预测高了还是低了，直接按照特征本身的数值比例去微调权重即可。

线性方法的收敛性 (Convergence of linear methods)

在线性情况下，误差曲面只有一个局部最优解（即**全局最优解**）。因此，任何保证能收敛到局部最优的方法，就等于保证能收敛到全局最优。例如，*蒙特卡洛梯度法*在线性函数逼近下就能收敛到均方价值误差 ($\overline{\text{VE}}$) 的全局最优点。

*半梯度 TD(0) 方法*在线性近似下也被证明是可以收敛的。但必须注意，**它不会收敛到 $\overline{\text{VE}}$ 的全局最优点（因为毕竟不是如完全采样比如MC获取的嘛，有运气的估计成分）**，

而是会收敛到另一个距离最优点有界限的点，这个点被称为 **TD 不动点 (TD fixed point, $\mathbf{w}_{TD}$)**。

以下证明推导：

为了证明 TD 算法最终会停在哪里，数学家把前面的更新公式改写成了矩阵/向量的形式：

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha(r_{t+1}\mathbf{x}_t - \mathbf{x}_t(\mathbf{x}_t - \gamma\mathbf{x}_{t+1})^T\mathbf{w}_t) \quad \text{— (8.15)}$$

这里的 $\mathbf{x}_t$ 就是状态 s_t 的特征向量。这段只是纯粹的代数展开。
如果系统运行了很久，达到了稳态，我们期望 $\mathbf{w}$ 不再发生剧烈变化。此时预期的权重更新公式可以简写为：

$$\mathbb{E}[\mathbf{w}_{t+1}|\mathbf{w}_t] = \mathbf{w}_t + \alpha(\mathbf{b} - \mathbf{A}\mathbf{w}_t) \quad \text{— (8.16)}$$

- $\mathbf{b}$：代表奖励和当前特征的乘积期望 $\mathbb{E}[R_{t+1}\mathbf{x}_t]$（一个向量）。
- $\mathbf{A}$：代表当前特征与“时间差分特征”的乘积期望 $\mathbb{E}[\mathbf{x}_t(\mathbf{x}_t - \gamma\mathbf{x}_{t+1})^T]$（一个矩阵）。

当系统收敛时，意味着“不再更新了”，即括号里的东西等于 0（$\mathbf{b} - \mathbf{A}\mathbf{w}_{TD} = 0$）。解这个方程得到：

$$\mathbf{w}_{TD} = \mathbf{A}^{-1}\mathbf{b}$$

$\mathbf{w}_{TD}$ **叫做 TD 不动点 (TD fixed point)**。这就是半梯度 TD 算法最终会收敛到的宿命位置！它不是碗底，但它是一个极其稳定的点。

在这个不动点上，误差的上限是：

$$\overline{\mathrm{VE}}(\mathbf{w}_{TD}) \leq \frac{1}{1-\gamma} \min_{\mathbf{w}} \overline{\mathrm{VE}}(\mathbf{w}) \quad \text{— (8.19)}$$

这就是我们上一轮聊过的误差上界。TD 找到的这个“不动点”虽然不是最完美的（右边的 $\min \overline{VE}$ 才是碗底），但最差也就比完美情况放大 $\frac{1}{1-\gamma}$ 倍。

为什么还要用 TD？ 虽然 γ 接近 1 时这个倍数可能很大（渐进性能有损失），但 **TD 方法的方差远小于蒙特卡洛方法，因此收敛速度快得多**！这就是在现实中大家依然偏爱 TD 的原因——它虽然有点偏，但学得极快。

碗状误差曲面：线性模型的误差地形就像一个极其平滑的碗，只要你顺着坡往下走（梯度下降），最后一定会停在唯一的碗底（全局最优）。没有坑坑洼洼的局部陷阱。

蒙特卡洛 vs. TD(0) 的终点之争：
- **蒙特卡洛 (MC)** 就像是一个“完美主义者”，它会准确无误地走到碗底的绝对最低点。
- **半梯度 TD(0)** 因为用了“半梯度”（自举法，带有一定的偏差），它走不到绝对的最底部，而是会停在碗底附近的一个点（**TD 不动点**）。

线性方法的特征构造 (Feature construction)

既然线性模型 $\hat{v}(s, \mathbf{w}) = \mathbf{w}^T\mathbf{x}(s)$ 这么简单，它怎么能处理复杂的世界呢？秘密就在于 **特征 $\mathbf{x}(s)$ 是可以由我们人工构造的**。

通俗比喻：线性模型就像一个只会做加法的老实人。如果你把原始的经纬度直接给他，他算不出什么有用的东西。但如果你提前帮他把经纬度转换成“距离市中心的公里数”、“是否在商圈内（0或1）”喂给他，他就能极其精准地算出这套房子的房价。这就叫特征构造！

使用线性方法时，为状态找到有效的**特征表示 (feature representation)**是至关重要的。在现实中，状态通常是由数字表示的（比如车辆的位置和速度），但这些原始数字直接用来做线性学习往往效果不好。常见的做法是**通过数学变换，将它们转化为新的向量表示**。

- **多项式 (Polynomials)**
  - **原理**：将原始状态元素的各阶幂以及它们的交叉乘积作为特征。
  - **例子**：假设原始状态只有两个数字 $s = (s_1, s_2)$（比如二维坐标）。构造出的 2 阶多项式特征向量就是：$\mathbf{x}(s) = (1, s_1, s_2, s_1^2, s_2^2, s_1 s_2)^T$
  - **优点与缺点**：它能捕捉到变量之间的**非线性交互作用**（比如 $s_1 s_2$）。但缺点是，如果原始维度很高，或者多项式阶数变高，特征维度会呈指数级爆炸（维度灾难）。
- **傅里叶基 (Fourier Basis)**
  - **原理**：用正弦和余弦函数提取原始状态的不同频率分量。
  - **通俗解释**：如果你在研究的问题带有周期性（比如钟摆的角度、一天中的时间），把原始数字转换成各种频率的波形特征，能让模型更好地理解规律。
- **砖瓦编码 / 网格编码 (Tile Coding)**
  - **原理**：将连续的空间（如 2D 或高维网格）划分成多个重叠的“砖瓦（网格）”。如果当前状态落在某个砖瓦内，该砖瓦对应的特征值就是 1，否则是 0。
  - **通俗解释**：想象你在一张地图上铺了好几层不同大小的渔网。你现在站在某个位置，你只需要大喊：“我踩中了第一层网的 A 格，第二层网的 B 格”。模型就把 A 和 B 对应的权重加起来。这在早期的强化学习（尤其处理 2D/3D 物理位置信号时）极其常用且高效。
- **径向基函数 (Radial Basis Functions, RBF)**
  - **原理 (公式 8.20)**：围绕预先设定的几个中心点 $\mathbf{c}_i$，测量当前状态 s 距离这些中心的远近。距离越近，特征值越大（呈现高斯钟形分布）。

$$x_i(s) = \exp\left(-\frac{||s - \mathbf{c}_i||^2}{2\sigma_i^2}\right)$$

- **通俗解释**：我们在地图上建了几个“信号塔”（中心点 $\mathbf{c}_i$）。这个特征代表的是 **“你当前位置接收到第 i 个信号塔的信号强度”**。你离它越近，这个特征值就越接近 1；离得远，就迅速衰减到 0。这在处理非线性特征时非常有效。

值得注意的是，线性形式本身**无法考虑特征元素之间的相互作用**。如果状态中某些元素之间的交互作用很重要，我们的策略必须是通过上述特征构造方法（比如多项式里的 $s_1 s_2$），把这种交互作用转化为专门的特征元素。

使用神经网络的非线性函数逼近 (Nonlinear Function Approximation with Neural Networks)

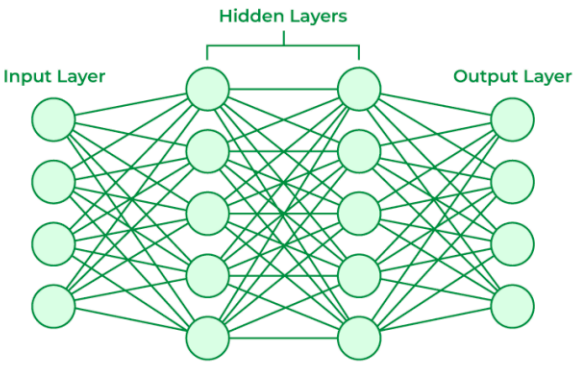

Figure 8.1: This figures illustrates the architecture of a simple neural network.

线性函数（把特征乘上权重再相加）的表达能力是非常有限的。在现实世界中，事物之间的关系往往错综复杂，线性模型无法捕捉这些复杂的、非线性的模式。

神经网络 (Neural networks) 被广泛应用于非线性函数逼近。一个神经网络由多层计算组成，每一层通常被公式化为一个线性变换，紧接着是一个非线性激活函数。
图 8.1 展示了一个简单的神经网络，它包含一个**输入层 (input layer)**，两个**隐藏层 (hidden layers)**，以及一个**输出层 (output layer)** 。具体来说，令 $\mathbf{x}$ 表示输入，那么计算过程可以写成

$$\mathbf{h}_1 = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) \quad \text{— (8.21)}$$

$$\mathbf{h}_2 = \sigma(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2) \quad \text{— (8.22)}$$

$$\mathbf{y} = \mathbf{W}_{out}\mathbf{h}_2 + \mathbf{b}_{out} \quad \text{— (8.23)}$$

- **为什么要用神经网络？** 想象你要用一条笔直的尺子（线性模型）去描绘一座连绵起伏的山脉，这显然是不可能的。神经网络就像是一把“可弯曲的百变尺”，能够拟合任意复杂的曲线和曲面。
- **公式剖析（以第一层为例）：**
  - $\mathbf{x}$：输入状态（比如一张图片的所有像素，或者游戏角色的坐标）。
  - $\mathbf{W}_1\mathbf{x} + \mathbf{b}_1$：这就是上一节学的**线性变换**。$\mathbf{W}$ 是权重矩阵，$\mathbf{b}$ 是偏置（截距）。
  - σ：**激活函数 (Activation function)**。这是神经网络的灵魂所在。
  - $\mathbf{h}_1$：第一隐藏层的输出（也叫特征/表示）。
- **通俗举例（流水线工厂）：** 神经网络就像一个有多道工序的加工厂。原材料 $\mathbf{x}$ 进入第一道工序（第一层），工人师傅用特定比例混合（线性变换 $\mathbf{W}$ 和 $\mathbf{b}$），然后放进烤箱里烤一下引发化学反应（非线性激活 σ），产出了半成品 $\mathbf{h}_1$。半成品再进入下一道工序...最后加工出成品 $\mathbf{y}$（价值预测）。

激活函数与反向传播

在这里，函数 σ 是一个**非线性激活函数 (nonlinear activation function)**，它以逐元素 (element-wise) 的方式应用于前面线性变换的输出上。在实践中使用了各种各样的激活函数。常见的选择包括定义为 $\sigma(x) = 1/(1+e^{-x})$ 的 *sigmoid 函数*，以及定义为 $\sigma(x) = \max(0, x)$ 的 *ReLU*。
梯度的计算可以遵循链式法则，从输出层逐层推导回输入层。这种计算可以被组织成一个系统化的算法，称为**反向传播 (back-propagation)**。

非线性激活函数 σ 的作用：如果不用 σ，无论叠多少层，数学上都可以证明它们最终会合并成一个单层线性网络（$W_2(W_1x) = (W_2W_1)x = W_{new}x$）。激活函数打破了这种线性叠加，给模型注入了“弯曲”的能力。

- **Sigmoid**：把任何数字压缩到 0 到 1 之间。
- **ReLU**：遇到负数直接变 0，正数原样输出。看似简单，但目前极其好用。

反向传播 (Back-propagation)：这是更新神经网络权重的核心方法。

- **通俗举例**：产品 $\mathbf{y}$ 做出来后，质检员发现太重了（计算出误差）。厂长（算法）顺着流水线往回找责任人："最后一道工序，你少加点料；上一道工序，你也稍微减一点..." 这种**把误差从后往前一层层分配**的过程，就是反向传播。

拓展：处理图像的卷积操作 (CNN)

当输入是 2D 或 3D 信号（例如图像）时，通常使用**卷积操作 (convolutional operation)** 来代替全连接的线性变换 。这利用了计算的局部性和平移不变性，极大地减少了参数数量和计算复杂度。
(通俗地说：看图时，不需要每个神经元都盯着整张图，而是像拿放大镜一样，一块一块地扫过图片提取特征。)

最小二乘 TD (Least-Squares TD, LSTD)

我们退回到**线性函数逼近**的场景。之前我们用的是随机梯度下降 (SGD)，也就是走一步看一步，慢慢逼近最优解。既然是线性问题，数学上存在一个绝对的终点（TD 不动点）。我们能不能不走弯路，直接列个方程组，用数学方法"空降"到终点呢？**LSTD** 就是干这个的。

直接求解 TD 不动点

带有线性函数逼近的 TD(0) 会收敛到如下的 *TD 不动点*：

$$\mathbf{w}_{TD} = \mathbf{A}^{-1}\mathbf{b}, \tag{8.24}$$

where

$$\mathbf{A} \triangleq \mathbb{E}\left[\mathbf{x}_t(\mathbf{x}_t - \gamma\mathbf{x}_{t+1})^T\right], \quad \text{and} \quad \mathbf{b} \triangleq \mathbb{E}\left[R_{t+1}\mathbf{x}_t\right]. \tag{8.25}$$

- $\mathbf{w}_{TD}$：我们梦寐以求的最终最优权重向量。
- $\mathbf{b}$：奖励与特征的结合体。大白话就是"在状态 $\mathbf{x}_t$ 下，平均能拿到多少奖励"。
- $\mathbf{A}$：当前特征 $\mathbf{x}_t$ 与"时序差分特征" $(\mathbf{x}_t - \gamma\mathbf{x}_{t+1})$ 的相关性矩阵。
- $^{-1}$：矩阵的逆。相当于标量运算里的"除法"。$\mathbf{w} = \mathbf{A}^{-1}\mathbf{b}$ 就像是求解 $ax = b$ 得到 $x = b/a$ 一样。

在 TD(0) 方法中，我们通过一个迭代更新过程来计算这个解。这里自然会产生一个问题：既然我们已经有了上面给出的**解析解 (analytic solution)**，我们能不能直接计算这个解，而不是走迭代的过程？答案是肯定的。**最小二乘 TD 算法 (Least-Squares TD, 简称 LSTD)** 做的正是这件事。

概念解析：
• **解析解**：就是能直接用公式算出来的准确答案（比如求一元二次方程的根公式）。与之相对的是迭代解（瞎猜一个数字，慢慢往正确答案上靠）。
• $\mathbf{A}$ 和 $\mathbf{b}$：它们其实是代表环境动态特性和奖励机制的统计矩阵。$\mathbb{E}$ 代表期望（理论上的完美平均值）。

通俗举例：
• **SGD（迭代法）**：你蒙上眼睛在山坡上往下走，每次用脚探一探哪边低就往哪边迈一步，最后走到谷底。
• **LSTD（解析法）**：你拿到了这座山的三维地形图（也就是收集到了 $\mathbf{A}$ 和 $\mathbf{b}$ 的数据），直接用超级计算机算出了谷底的 GPS 坐标（$\mathbf{A}^{-1}\mathbf{b}$），然后坐直升机直接降落在那里。

LSTD 的计算与增量更新 (Sherman-Morrison 公式)

具体来说，LSTD 形成了以下自然估计（用收集到的实际数据代替期望）：

$$\hat{\mathbf{A}}_t = \sum_{k=0}^{t-1} \mathbf{x}_k(\mathbf{x}_k - \gamma \mathbf{x}_{k+1})^T + \epsilon \mathbf{I}, \quad \text{and} \quad \hat{\mathbf{b}}_t = \sum_{k=0}^{t-1} R_{k+1}\mathbf{x}_k \quad \text{— (8.26)}$$

- **从期望到经验估计 (8.26)**：理论上的期望值 $\mathbb{E}$ 我们算不出来，所以我们让智能体去跑游戏，收集数据。然后把每一步收集到的数据直接累加起来求和（$\sum$），得到 $\hat{\mathbf{A}}_t$ 和 $\hat{\mathbf{b}}_t$。
- 为什么要加 "？：在数学中，如果一个矩阵的数据太少或者共线，它是无法求逆的（相当于分母为 0）。加上一丁点微小的常数矩阵 "，相当于给它加了个保险，保证计算不崩溃。

其中 $\mathbf{I}$ 是单位矩阵，加上一个极小的 $\epsilon\mathbf{I}$ 是为了确保 $\hat{\mathbf{A}}_t$ 总是可逆的。然后 LSTD 直接计算 TD 不动点：

$$\mathbf{w}_t = \hat{\mathbf{A}}_t^{-1}\hat{\mathbf{b}}_t \quad \text{— (8.27)}$$

- **帽子符号**^：在统计学里，戴帽子意味着"**估计值**"。
- **通俗推导**：公式 8.24 里有 $\mathbb{E}$（真实的概率期望），但这需要上帝视角。现实中，我们只能让 AI 去环境里跑，收集一条条经验。我们把收集到的前 t 步数据求个平均，算出一个**经验矩阵** $\hat{\mathbf{A}}_t$ 和**经验向量** $\hat{\mathbf{b}}_t$。只要数据足够多，戴帽子的估计值就会无限接近真实值。

这个算法是线性 TD(0) **数据效率最高**的形式，但它的**计算代价也更加高昂**。一般来说，构造 $\hat{\mathbf{A}}_t$ 的复杂度是 $O(d^2)$，但是计算矩阵逆 $\hat{\mathbf{A}}_t^{-1}$ 的复杂度是 $O(d^3)$（d 是特征维度）。
幸运的是，因为 $\hat{\mathbf{A}}_t$ 被公式化为外积的和，所以可以使用 **Sherman-Morrison 公式**对其逆矩阵进行增量更新：

$$\hat{\mathbf{A}}_t^{-1} = \hat{\mathbf{A}}_{t-1}^{-1} - \frac{\hat{\mathbf{A}}_{t-1}^{-1}\mathbf{x}_t(\mathbf{x}_t - \gamma\mathbf{x}_{t+1})^T\hat{\mathbf{A}}_{t-1}^{-1}}{1 + (\mathbf{x}_t - \gamma\mathbf{x}_{t+1})^T\hat{\mathbf{A}}_{t-1}^{-1}\mathbf{x}_t} \quad \text{— (8.28)}$$

这里的计算复杂度降低到了 $O(d^2)$。

就比如我们每走一步收集到一个新数据，$\hat{\mathbf{A}}$ 矩阵就会改变一点点。我们要算 $\mathbf{w}$，就必须对新的 $\hat{\mathbf{A}}$ **求逆矩阵**。

在数学中，求逆矩阵是一个**计算量极其庞大**的操作（复杂度是特征维度的三次方）。如果每走一步都要重新求一次逆，电脑早就卡死了。

通俗比喻：假设你每天都在算过去所有日子开销的平均值。今天花了一笔新钱，你是把过去 10 年的账本全部翻出来重新加一遍除以总天数？还是直接用“昨天的平均值”加上一个“今天的调整量”？显然是后者！公式 8.28 (即谢尔曼-莫里森公式) 就是矩阵界的“增量更新法则”。**它允许我们直接用上一步的逆矩阵 $\hat{\mathbf{A}}_{t-1}^{-1}$，通过简单的加减乘除，瞬间算出今天的逆矩阵 $\hat{\mathbf{A}}_t^{-1}$**，完美避开了重新求逆的恐怖计算量。

Sherman-Morrison 公式的伟大之处 (8.28)：
- **痛点**：计算矩阵的逆矩阵（类似倒数）极其消耗算力，尤其是矩阵很大时。如果智能体每走一步，你都要把过去的几十万条数据重新组合算一次逆矩阵，电脑会卡死（$O(d^3)$ 复杂度）。
- **解法**：Sherman-Morrison 公式提供了一个魔法：当你有了一个新的数据点，你**不需要重新求逆**，只需要在**上一步已经求好的逆矩阵 $\hat{\mathbf{A}}_{t-1}^{-1}$** 的基础上，做一点简单的加减乘除，就能瞬间得到新的逆矩阵。这把计算量大降级（降到了 $O(d^2)$），让 LSTD 在实际中变得可用。
- **生活类比**：你算出了前 100 场考试的平均分。第 101 场成绩出来时，你不需要把 101 个数字重新加一遍再除以 101。你只需要用原来的平均分做个简单的换算就行了。Sherman-Morrison 就是矩阵界做这种换算的法则。

关键概念解释

期望 vs 概率分布的区别是什么？

- **概率分布 (μ)** 是一张“清单”。比如掷骰子：{1: 1/6, 2: 1/6, 3: 1/6, 4: 1/6, 5: 1/6, 6: 1/6}。
- **期望 ($\mathbb{E}$)** 是一个基于清单算出来的“加权平均数值”。骰子的期望是：$1 \times (1/6) + 2 \times (1/6) ... + 6 \times (1/6) = 3.5$。
- **所以，期望不是用来“替代”分布的，期望是“使用”分布算出来的一个结果。**

μ 是什么形状的？怎么算出具体数值的？
- **形状**：μ 是一个**向量**（可以理解为一维矩阵）。如果迷宫有 100 个格子，μ 就是一个包含 100 个小数的数组，加起来等于 1。$\mu(s_5) = 0.02$ 是一个具体的数值，表示“AI 有 2% 的概率呆在 5 号格子”。
- **魔法就在这里：在实际编程中，我们根本不需要把 μ 给算出来！**

凭什么不计算就能算出期望？（大数定律的魔法）
你想计算掷 100 次骰子的平均点数（期望），你有两种方法：
- **书呆子方法（依赖 μ）**：你在纸上列出每个点数的概率 μ，严格按照公式算期望，得出 3.5。
- **实干家方法（强化学习的做法）**：你根本不管什么概率分布。你直接拿起骰子扔 10000 次，把所有扔出来的点数加起来除以 10000。结果一定是极其接近 3.5 的！

在强化学习里，公式 $\mathbb{E}_{s \sim \mu}[...]$ 的意思是：“在这个概率分布下求平均”。
我们的做法就是实干家做法：**让 AI 直接去环境里打游戏！**
AI 经常去的格子，自然产生的训练样本就多；AI 去不到的格子，产生的样本就少。**只要我们把 AI 玩游戏产生的一条条经验直接塞进公式（图 4 的 8.14）里更新参数，这个过程在数学上就已经等价于“乘以了概率分布 μ 并求了期望”！**
这就是强化学习最美妙的代数技巧：用**环境的真实交互采样 (Sampling)**，完美代替了**极其复杂的概率积分运算**。